capcut-cli：让 Agent 在终端里剪视频，剪映里继续手动调

直接读写剪映/CapCut 本地草稿 JSON——"JSON in, JSON out"，无 API、无守护进程、轨道全部保持可编辑

GitHub 966★ | MIT | Node ≥18 零依赖 | 实测 v0.31.1

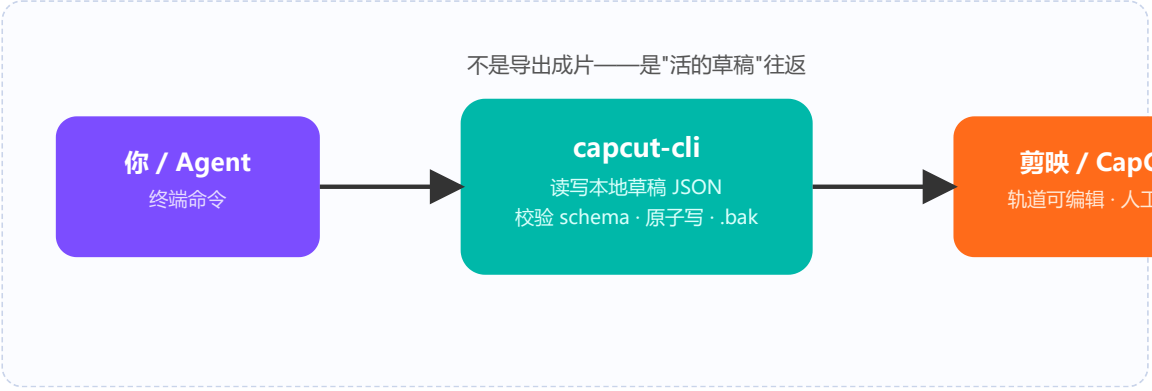
仓库：github.com/renezander030/capcut-cli ｜ 非官方，与字节跳动无关 ｜ 学习日期 2026-10-27

Q1

capcut-cli 是什么？一句话能说清吗？

一个非官方的剪映命令行：不调用任何 CapCut API，直接读写剪映存在本地的草稿 JSON 文件，让 Agent 或你可以在终端里完成建草稿、加素材、剪静音、上字幕、做翻译，产出的草稿打开剪映后每条轨道都还是活的、能接着手动精调。

它不是"另一个剪辑软件"，而是**给剪映外挂的自动化前道**：Agent 批量干粗活（自动剪空白、打字幕、出多语言版本），人继续在剪映里做精修。官方 README 的定位句："JSON in, JSON out: every command reads and writes the local draft store directly, with no MCP server or HTTP daemon."



Q2

为什么需要它？Agent 剪辑的痛点是什么？

视频是 Agent 时代最难自动化的媒介：剪辑软件全是 GUI，没有 API。

Agent 想"把这段口播的空白剪掉、打上字幕"，面前只有两条烂路：① 模拟点击剪映界面（脆弱、慢）；② 用 FFmpeg 硬剪（产出是死视频，人没法再调）。

capcut-cli 指出了第三条路：**剪映的草稿本来就是 JSON。**既然格式是开放的，就直接读写它——

方案	产出可人工精修	稳定性	批量自动化
GUI 自动化（点剪映）	✅	❌ 界面一变就崩	❌
FFmpeg 硬剪	❌ 死视频	✅	✅
capcut-cli 改草稿	✅ **轨道全活**	✅ **JSON 确定性**	✅ **命令可脚本化**

这正好接上知识库里 [[声明式视频源语言]] / [[分镜表驱动生成]] 的路线：视频生产越来越"结构化中间表示驱动"，剪映草稿 JSON 就是其中一种工业级 IR。

Q3

它是怎么做到的？核心机制是什么？

① 草稿即 JSON：直接读写本地草稿库

剪映把每个工程存在草稿目录（`draft_content.json` / `draft_info.json`）。capcut-cli 每条命令 = 加载草稿 → 按版本感知 schema 校验 → 应用编辑 → **原子写回 + .bak 备份**。实测草稿目录里每个 JSON 都自动带同名 .bak。

② 版本感知 schema

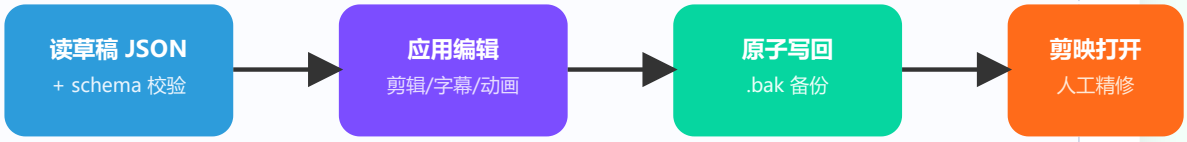
不同剪映版本的草稿格式不同（支持 6.x–9.x）。CLI 内置模板与 schema 探测，`version` 命令报告版本标记与 schema 特性；新版多时间线（CapCut 8.7 的 mirror 结构）用 `sync-timelines` 对齐修复，不假定 draft_content.json 是唯一真值。

③ 安全写

默认拦截"剪映正在运行/磁盘上已被改过/版本越界"的写入（`--force-write` 才覆盖）；`--dry-run` 预览不动盘；`restore` 凭 .bak 撤销。还有 `doctor` 环境体检与 `lint` 草稿体检（实测 0 错误 0 警告通过）。

④ 可选能力外置

核心零依赖；Whisper（字幕）、FFmpeg（预览渲染）、ffprobe（媒体时长）、翻译 API（多语言克隆）都是可选外挂——本机无 ffmpeg 时图片素材自动用 5 秒占位并明确告知，不偷偷瞎编。



Q4

关键术语都是什么意思?

术语	一句话解释
草稿 (draft)	剪映工程在本地的一组 JSON 文件（draft_content.json 等）
轨道 (track) / 片段 (segment)	时间线上的层（视频/音频/文字轨）和层上的素材区间
draft_materials	草稿的素材登记表：所有被引用的媒体在这里登记
schema 探测	读草稿里的版本标记，判断该用哪套格式规则读写
原子写 + .bak	先写临时文件再切换，同时保留改前备份，崩了可 restore
--dry-run	预览写操作结果但不落盘，.bak 也不动
lint	草稿体检：按版本规则查错误/警告（如 JianYing 11.6 的 UNIFORM_SCALE 检查）
quickstart / compile	一条命令建首个草稿 / 从 JSON spec 批量构建草稿
render（低分辨率代理）	FFmpeg 出低清预览，**不是**剪映的最终导出

Q5

它能干什么？命令全景

组	代表命令	用途
查看	info / tracks / materials / version / lint	摸清水草稿结构
创建	init / quickstart / compile	建草稿；compile 从 JSON spec 批量构建
加料	add-video / add-audio / add-text	加素材与文字（支持 Wikimedia URL）
精剪	trim / speed / volume / 转场 / 蒙版 / 动画 / 缓动	片段级编辑
字幕与多语言	caption / import-srt / export-srt / translate	Whisper 自动字幕；克隆多语言版本
长视频→shorts	detect-silence / cut / detect-scenes / detect-retakes	按静音/场景自动切块
兜底	doctor / sync-timelines / restore / migrate	体检 / 修 8.7 mirror / 撤销 / 迁移

Agent 集成有两条路：装 `capcut-edit` skill（`npx skills add`）让 Agent 学会命令；或实验性的 Wasm 组件（只读 inspect/diff/lint 三个 MCP 工具，零文件零网络权限）。

Q6

它和已有概念是什么关系?

已有概念	本项目中的体现
[[JSON-IR类型化中间表示]]	剪映草稿 JSON 就是视频生产的 IR：Agent 只改语义表，渲染交给剪映
[[声明式视频源语言]] / [[分镜表驱动生成]]	同路线家族：compile 命令 = 把 JSON spec"编译"成草稿
[[事实与判断分离]]	版本探测是确定性代码干的；剪辑决策可以交给 Agent
[[质检Gate与自我纠错循环]]	lint 门禁 + doctor 体检 + dry-run 预览 = 写盘前三道 Gate
[[Checkpoint存档与持久执行]]	.bak 备份 + restore 撤销 = 草稿级的 checkpoint 机制
[[产物留痕与状态外置]]	草稿 JSON 就是落盘产物，Agent 崩溃后下一位从 JSON 接着干

Q7

实测：无剪映环境能跑到哪一步？

本机实验（详见 `capcut_cli\exercise\experiment-log.md`）：**没有安装 CapCut、没有 ffmpeg**，用 Python 标准库自造了一个 320×240 PNG 和一段 2 秒 WAV：

```
# 一条命令建草稿（图片无时长 → 明确用 5 秒占位并告知）
capcut quickstart demo1 --video clip.png --audio bgm.wav --ratio 16:9
# → create ✓ / add-video ✓(5s占位) / add-audio ✓(5.00s) / lint ✓ 0警告

capcut tracks demo1        # → #0 video 5.00s / #1 audio 5.00s
capcut add-text demo1 0 2 "hello capcut" --color "#FF2E88"
                           # → 新增 #2 text 2.00s 独立轨道
capcut trim demo1 <id> 0 1 --dry-run   # → {"dryRun":true} 不落盘
capcut lint demo1          # → {"ok":true,"errors":0,"warnings":0}
```

> **实测结论**：零外部依赖链路（建草稿→查轨道→加文字→dry-run→lint）全部真实跑通；每一步都留 .bak；缺 ffprobe 时诚实降级为占位而不是瞎编时长。

Q8

边界与风险是什么？

- **最终导出永远在剪映**：CLI 的 render 只是低清预览代理，成片导出要回剪映按；
- **版本敏感**：格式跟剪映版本走（支持 6.x–9.x），剪映大版本升级后要先 `version` + `sync-timelines` 核对；
- **旧版本有安全污点**：官方自述 0.18.0 之前存在本地命令/过滤器注入路径与不安全临时文件行为，≤0.17.2 的 fixtures 可能含稳定设备标识符——**务必用最新版**；
- **云字幕是实验性**：caption --cloud 会把音频传到字节服务器，非官方且涉及上传，生产环境慎用；
- **部分路径未验证**：桌面版对"剪映自建工程"的持久化写入官方标注"remains unverified"；
- **第三方工程**：与字节跳动无任何关系，格式逆向而来，理论上剪映改格式就会追修。

Q9

"Agent 剪视频"这个场景值不值得学?

值得，因为它示范了一种通用架构：把"GUI 软件"变成"文件格式接口"。

大量专业软件（剪辑、CAD、音频工作站）都没有 API，但都有**工程文件**。capcut-cli 的路线——逆向工程文件格式 + 版本感知 schema + 原子写——可以平移到任何这类软件上。对 Agent 工作流而言，这相当于把"只能人操作的软件"变成了"可编程的资源"。

它与 [[声明式视频源语言]]（hypit）的区别在定位：hypit 想替代整条视频生产线，capcut-cli 则是**寄生在现有工业软件上的自动化层**——落地阻力小得多，今天就能用。

Q10

实际怎么装、怎么用?

```
# 安装（三选一）
npm install -g capcut-cli     # 全局
npx capcut-cli <command>      # 一次性
pip install capcut            # Python 包装：capcut.run(...)

# 日常闭环
capcut doctor                 # ① 环境体检
capcut quickstart mydraft --video a.mp4 --srt sub.srt   # ② 一条命令建草稿+字幕
capcut detect-silence mydraft --cut   # ③ 自动剪静音
capcut tracks mydraft -H      # ④ 核对轨道
# ⑤ 打开剪映人工精修 → 导出成片
```

> 给 Agent 用：`npx skills add` 装 capcut-edit skill，Agent 即获得全部命令的用法与约束；Wasm 实验组件提供零权限只读三件套（inspect/diff/lint），适合"先看再说"的审计场景。

这个项目给我什么能力?

1. **口播视频自动化**：一段原始口播 → 自动剪静音 + Whisper 字幕 → 打开剪映微调即发布；
2. **多语言批量出片**：translate 克隆多语言草稿，一次录制全球分发；
3. **长视频切 shorts**：detect-scenes/detect-silence 自动切块，矩阵号内容流水线；
4. **给 Agent 装"剪辑手"**：配合 [[分镜表驱动生成]]，从选题到草稿全自动，人只做最后审美；
5. **学"文件格式自动化"范式**：任何无 API 的 GUI 软件，都可复制"读写工程文件"这条路线。

动手

实验：10 分钟建一个真实剪映草稿（无剪映也能跑）

步骤 1 ｜ 构建。

```
git clone --depth 1 https://github.com/renezander030/capcut-cli repo
cd repo && npm install && npm run build
node dist/index.js --version    # → 0.31.x
```

✅ 预期结果：版本号输出。（bin 入口是 dist/index.js。）

步骤 2 ｜ 自造测试素材。用任意方式准备一张图片和一个音频文件（本实验用 Python 标准库生成了 320×240 PNG 与 2 秒 WAV）。

✅ 预期结果：clip.png 与 bgm.wav 就位。

步骤 3 ｜ quickstart 建草稿。

```
node dist/index.js quickstart demo1 --video clip.png --audio bgm.wav --ratio 16:9 --dr
```

✅ 预期结果（本机真实输出）：create ✓（注册进工程索引、画布 1920x1080）→ add-video ✓（无 ffprobe 用 5s 占位并明示）→ add-audio ✓（5.00s）→ lint ✓ 0 警告。

步骤 4 ｜ 看轨道、加文字。

```
node dist/index.js tracks drafts/demo1 -H
node dist/index.js add-text drafts/demo1 0 2 "hello" --color "#FF2E88"
node dist/index.js tracks drafts/demo1 -H    # 多出 #2 text 2.00s
```

✅ 预期结果：文字作为独立轨道出现，返回 segment_id/material_id/track_id。

步骤 5 ｜ 安全机制体验。

```
ls drafts/demo1            # 每个 JSON 都带 .bak
node dist/index.js trim drafts/demo1 <id> 0 1 --dry-run   # dryRun:true 不落盘
node dist/index.js lint drafts/demo1                       # 0 错误 0 警告
```

✅ 预期结果：备份文件齐全；dry-run 只预览；lint 干净。

如实声明：本机未装 CapCut，"打开剪映人工核验轨道可编辑"一步未验证；caption/render 需 Whisper/FFmpeg 亦未验证。其余链路全部真实执行，日志见 `capcut_cli\exercise\experiment-log.md` 。

一页总结

- **它是剪映草稿的 JSON 读写器**：无 API、零依赖、轨道保持可编辑，Agent 干粗活、人做精修；
- **工程纪律是亮点**：版本感知 schema、原子写 + .bak、dry-run、lint 门禁，写盘前三重保护；
- **可选能力外置**：Whisper/FFmpeg/翻译 API 都是外挂，缺了诚实降级不瞎编；
- **边界要清醒**：最终导出回剪映；旧版有安全污点务必升级；云字幕会外传音频；
- **范式可平移**："读写工程文件"是让一切 GUI 软件可被 Agent 操作的通用路线。